Perfil técnico: Guardianes del Chaco

· @Touken_4

Proyecto Integrador de Computación Paralela y Distribuida · Hito 1 · Estudiante: [tu nombre] · Docente: [nombre del docente]

1. Problema regional

Bolivia vivió en 2024 su peor temporada de incendios, con 12,6 millones de hectáreas quemadas según Fundación Tierra. En julio de 2026, Tarija declaró emergencia departamental por incendios que ya habían afectado unas 5.000 hectáreas (EFE, vía Infobae). Decidir dónde abrir cortafuegos o ubicar brigadas exige estimar cómo avanza el fuego con distintos vientos, humedades y relieves, y una sola simulación no basta: hacen falta miles.

2. Solución propuesta

«Guardianes del Chaco» es un simulador de propagación de incendios con interfaz de juego por turnos, en pixel art y sin violencia. La jugadora coloca cortafuegos, pozos y brigadas para proteger el bosque y guiar a los animales del Chaco hasta sus refugios. El clúster es el motor: simula el fuego y ejecuta el «Vigía», un oráculo Monte Carlo que estima la probabilidad de quema de cada zona.

3. Modelo matemático

Autómata celular estocástico en una grilla N×N con vecindad de Moore. Una celda sin quemar se enciende con probabilidad $p = 1 - \Pi(1 - p_n)$, donde cada vecino en llamas aporta $p_n = p_{base} \times (1 - h) \times f_{viento} \times f_{pendiente} \times f_{diag}$, con h la humedad de la celda. El número aleatorio de cada celda es $u = hash(seed, paso, celda)$, así que las versiones secuencial y paralelas producen resultados idénticos. El Vigía repite M simulaciones y promedia las celdas quemadas para obtener la probabilidad de quema por celda.

4. Volumen computacional

Un mapa de juego de 200×200 celdas con 100 pasos y 2.000 simulaciones del Vigía suma unas $8 \cdot 10^9$ actualizaciones de celda. Los mapas de benchmark llegan a 8192^2 celdas. Ese volumen justifica la paralelización.

5. Arquitectura paralela

Capa	Tecnología	Función
Memoria compartida	OpenMP en C	Paralelismo de datos en cada nodo: doble buffer, `reduction` y `schedule`
Memoria distribuida	OpenMPI en C	Partición por filas con halo (`Isend`/`Irecv`), `Scatter`, `Gather` y `Allreduce`; Monte Carlo repartido y reunido con `Reduce`
Híbrido	OpenMP + MPI	Un proceso MPI por nodo con varios hilos

Clúster	Docker Compose (Opción A)	1 master y 2 workers, red bridge, SSH sin contraseña, OpenMPI y GCC en contenedores
Cliente	Python + pygame	Visualización y control; consulta al motor por TCP

Taxonomía de Flynn: MIMD, con memoria distribuida entre nodos y memoria compartida dentro de cada nodo.

6. Plan de evaluación

Se medirá la versión secuencial (T1) y la paralela (Tp) con 1, 2, 4, 8 y 16 hilos, 1, 2 y 4 nodos o procesos, y tamaños de problema pequeño, mediano y masivo. Métricas: Sp = T1/Tp y Ep = Sp/p, con ajuste por la Ley de Amdahl (escalabilidad fuerte) y la Ley de Gustafson (escalabilidad débil). Se verificará que la salida paralela sea idéntica a la secuencial. Las mediciones oficiales se harán en las Dell del laboratorio; las pruebas previas, en una laptop y en un equipo auxiliar (máquina virtual).

7. Cronograma

Hito 1 (semana 2): este perfil. Hito 2 (semana 5): clúster operativo en las Dell y código secuencial base. Hito 3 (semana 8): repositorio, informe IEEE y defensa. Trabajo individual.